DA3408 A3: Spark vs. Ray — The Data Engineering Duel

This assignment introduces the concept of **Unified Compute**, where you will contrast the industry-standard **Apache Spark** with the modern, AI-focused **Ray** framework. While Spark excels at structured data parallelism, Ray is designed for flexible task parallelism. Your goal is to determine which framework "cleans" data more efficiently in a cluster environment.

Objective: Implement an identical data preprocessing pipeline in both Apache Spark and Ray, deploy and benchmark their performance on a large-scale dataset.

1. The Data Engineering Task

You will work with a **large-scale, uncleaned dataset** (New York City Taxi Trip Data ~ 2GB).

Pipeline Steps:

1. **Ingestion:** Load multiple CSV/Parquet files from a shared directory.
2. **Cleansing:** Filter out null values, remove duplicates, and format timestamps.
3. **Transformation:** * Perform a "Heavy Join" between two tables (e.g., Trip data and Location IDs).
    - Calculate complex derived features (e.g., "Average Speed per Hour" using a custom Python function/UDF).
4. **Export:** Save the structured result to a Parquet file ready for ML training.

---

2. Infrastructure Setup

Each cluster must use **two workers** to form a distributed cluster.

- **Spark Setup:** Configure two `Workers` using **PySpark**.
- **Ray Setup:** Initialize a Ray cluster with two `Workers`. Use **Ray Data** (`ray.data`).

---

3. Comparison & Benchmarking

You must collect and report the following metrics for **both** frameworks:

- **Execution Time:** Total wall-clock time from ingestion to export.
- **Resource Utilization:** Peak CPU and Memory usage (use the `top` command or the Prometheus/Grafana stack from Assignment 5).
- **Python UDF Overhead:** Specifically measure how long the custom Python transformation takes in Spark (JVM overhead) vs. Ray (Python-native).

---

4. Code of Conduct: Fair Use of AI

- **The "Logic Translation" Rule:** You may use AI to help translate a PySpark snippet into Ray Data syntax (or vice versa). However, you must perform the **Cluster Configuration** and **Network Setup** manually.
- **No Synthetic Benchmarks:** You are strictly forbidden from using AI to "generate" performance numbers. We will verify your results by checking the Spark UI (Port 4040) and Ray Dashboard (Port 8265) screenshots.
- **Attribution:** If you use an LLM to optimize a specific transformation, you must include a "Performance Tuning Note" in your report explaining the AI's suggestion.

---

5. Grading Rubric (100 Points)

Category	Weight	Criteria
Cluster Orchestration	25	Successfully set up the cluster for both Spark and Ray. Evidence via UI screenshots.
Pipeline Parity	25	The preprocessing logic is identical across both frameworks. Results match exactly.
Performance Analysis	20	Clear, tabulated comparison of execution times and resource spikes.
UDF Deep-Dive	15	Insightful observation on the "Python-native" advantage of Ray vs. the JVM-based Spark.

Documentation	15	A concise report (3–5 pages) with visualizations of the benchmarks and a "Winner" conclusion.

Final Submission

1. **Codebase:** A GitHub repo with `spark_clean.py` and `ray_clean.py`.
2. **Screenshots:** * Spark Master UI showing 2 active workers.
    - Ray Dashboard showing the cluster resource view.
3. **Benchmark Report:** A PDF containing the performance graphs and a brief discussion on which framework you would choose for an AI-first vs. a BI-first project.

Reading Material

- **Spark vs. Ray:** https://aiwiki.ai/wiki/ray
- **Ray Data:** Getting Started with Distributed Data Loading